CEG5201 Hardware Technologies, Principles, & Platforms
(Semester I, AY2026/2027)
CA-2 Statement

Release date: 27th Sept 2026
Consultation date: 13th Oct 2026
Submission deadline: 31st Oct 2026 11:59pm

Instructions to candidates:

The CA2 Canvas submission deadline is midnight as shown above. No late submission is considered, try NOT to do last-minute submissions to prevent traffic congestion.

I. Objective

In this assignment, you will benchmark matrix multiplication algorithms using sequential and multiprocessing methods. Review the following Python multiprocessing material before beginning the work:

Official Python multiprocessing documentation:
https://docs.python.org/3/library/multiprocessing.html

Use Python's standard-library multiprocessing.Pool for the multiprocessing implementation. **Do not use a thread pool for the benchmarked matrix-multiplication computation.**

You may also consult other useful resources. Share useful new links on the Canvas Discussion Board so that they benefit the class.

Important Advisory on Python Multiprocessing Performance

1. NumPy operations can use multi-threaded BLAS/LAPACK backends, for example OpenBLAS and MKL.
2. NumPy or BLAS/LAPACK libraries may create internal threads automatically within each multiprocessing.Pool worker. This can oversubscribe CPU cores and reduce performance.
3. Implement the core matrix operations for the assigned algorithm yourself; do not use np.matmul(), numpy.dot(), or @ as the sequential benchmark.
4. If NumPy is used for array storage or other auxiliary work, set OMP_NUM_THREADS, MKL_NUM_THREADS, OPENBLAS_NUM_THREADS, VECLIB_MAXIMUM_THREADS, and

NUMEXPR_NUM_THREADS to 1 before importing NumPy. See https://superfastpython.com/numpy-processes-worse-performance/ for background.

II. Project overview

We will attempt to implement matrix multiplication through four different algorithms (**Strassen's algorithm** [1], **Coppersmith–Winograd algorithm** [2], **Cannon's algorithm** [3], and **Fox's algorithm** [4]) in this project, and each team member focuses on only one algorithm and implement in both sequential and multiprocessing versions. The project is composed of individual tasks and joint tasks as shown in Table I. Blue shaded parts in the table are individual tasks and yellow shaded parts are joint tasks.

Table I. Overview of the tasks

Task ID	**Task description**
A	Matrices generation
B-n	Sequential implementation
C-n	Multiprocessing implementation
D-n	Determining the ultimate speed-up
E	Comparison of the algorithms
F-n	Intuitive implementation walkthrough

For Tasks B-n, C-n, D-n, and F-n, use n = (Group ID + Member ID) % 4 + 1. Table II maps n to an algorithm. Refer to Appendix A for the general ideas behind the four algorithms.

Table II. List of the algorithms used

S/N (n)	**Algorithm**
1	Strassen's algorithm
2	Coppersmith–Winograd algorithm
3	Cannon's algorithm
4	Fox's algorithm

III. Task description

[O] Getting started

As a team, select one fixed multi-core CPU platform before individual coding. Report the processor and number of usable CPU cores, and use the same platform, matrix data, process-count range, workload order, and timing boundaries for all members' benchmarks.

[A] Matrix generation

Randomly generate **8 pairs** of matrices $(A_i,\ B_i),\ i \in [0, 1, \ldots 7]$ with $M_i \times M_i$ size where M= [16, 32, 64, 128, 256, 512, 1024, 2048]. The value of each element should

be between 0 and 255 (integers only). Generate 10 groups of such matrix pairs $G_j,\ j \in [0, 1, \ldots 9]$. All the team members need to use the same set of G for their subsequent tasks.

[B-n] Sequential Processing

[B-n.1] Implement the assigned algorithm sequentially for all eight matrix pairs in G_0. Record the cumulative elapsed time through each pair. Let t_i be the actual time for pair i and T_i be the cumulative elapsed time through pair i, where $T_i = \Sigma_{r=0}^{i} t_r$. Present the results in Table B-n.1.

Table B-n.1: Processing time of G_0 under sequential implementation

Pair index	Measured cumulative elapsed time	Derived time for this pair
0	T_0	$t_0 = T_0$
1	T_1	$t_1 = T_1 - T_0$
2	T_2	$t_2 = T_2 - T_1$
…	…	…
7	T_7	$t_7 = T_7 - T_6$

For $i \geq 1$, derive the time for pair i as $t_i = T_i - T_{i-1}$.

[B-n.2] Process all ten groups G_0 through G_9 sequentially. For each group, record its total processing time $T(G_i)$, then compute the cumulative time through that group. Present all group-level results in the single summary Table B-n.2.

Table B-n.2: Summary table for sequential processing of all groups

Group index	Measured total time for this group	Cumulative time through this group
0	$T(G_0)$	$T(G_0)$
1	$T(G_1)$	$T(G_0) + T(G_1)$
2	$T(G_2)$	$T(G_0) + \ldots + T(G_2)$
…	…	…
9	$T(G_9)$	$T(G_0) + \ldots + T(G_9)$

[C-n] Multiprocessing

[C-n.1] Identify the sequential bottleneck and accelerate it using multiprocessing. For G_0, repeat the benchmark for p = 1, …, min(6, available usable CPU cores) processes. At each p, record cumulative elapsed time through each pair, $T_i^{(p)}$, and derive the time for pair i, $t_i^{(p)}$.
State whether worker-pool creation is included in the timed region and apply that rule uniformly.

Table C-n.1: Processing time of G_0 under multiprocessing implementation

	Measured cumulative MP time $T_i^{(p)}$						**Derived MP time for pair $t_i^{(p)}$**					
Process p → Pair index i ↓	**1**	**2**	**3**	**4**	**5**	**6**	**1**	**2**	**3**	**4**	**5**	**6**
0	$T^{(1)}_0$	$T^{(2)}_0$	$T^{(3)}_0$	$T^{(4)}_0$	$T^{(5)}_0$	$T^{(6)}_0$	$t^{(1)}_0$	$t^{(2)}_0$	$t^{(3)}_0$	$t^{(4)}_0$	$t^{(5)}_0$	$t^{(6)}_0$
1	$T^{(1)}_1$	$T^{(2)}_1$	$T^{(3)}_1$	$T^{(4)}_1$	$T^{(5)}_1$	$T^{(6)}_1$	$t^{(1)}_1$	$t^{(2)}_1$	$t^{(3)}_1$	$t^{(4)}_1$	$t^{(5)}_1$	$t^{(6)}_1$
2	$T^{(1)}_2$	$T^{(2)}_2$	$T^{(3)}_2$	$T^{(4)}_2$	$T^{(5)}_2$	$T^{(6)}_2$	$t^{(1)}_2$	$t^{(2)}_2$	$t^{(3)}_2$	$t^{(4)}_2$	$t^{(5)}_2$	$t^{(6)}_2$
3	$T^{(1)}_3$	$T^{(2)}_3$	$T^{(3)}_3$	$T^{(4)}_3$	$T^{(5)}_3$	$T^{(6)}_3$	$t^{(1)}_3$	$t^{(2)}_3$	$t^{(3)}_3$	$t^{(4)}_3$	$t^{(5)}_3$	$t^{(6)}_3$
…	…	…	…	…	…	…	…	…	…	…	…	…
7	$T^{(1)}_7$	$T^{(2)}_7$	$T^{(3)}_7$	$T^{(4)}_7$	$T^{(5)}_7$	$T^{(6)}_7$	$t^{(1)}_7$	$t^{(2)}_7$	$t^{(3)}_7$	$t^{(4)}_7$	$t^{(5)}_7$	$t^{(6)}_7$

For each permitted process count p, $t_0^{(p)} = T_0^{(p)}$; for $i \geq 1$, derive the time for pair i as $t_i^{(p)} = T_i^{(p)} - T_{i-1}^{(p)}$.

Example (for any permitted process count p): the derived MP time for pair 2 is

$$t_2^p = T_2^p - T_1^p$$

[C-n.2] For every permitted process count p, process all ten groups G_0 through G_9. Record the total multiprocessing time $T^{(p)}(G_i)$ for each group and compute the cumulative multiprocessing time through that group. Present all group-level results in the single summary Table C-n.2.

Table C-n.2: Summary table for multiprocessing of all groups

	Measured MP time for group $T^p(G_g)$						**Cumulative MP time through group $\Sigma_{r=0}^{g} T^p(G_r)$**					
Process p → Group index g ↓	**1**	**2**	**3**	**4**	**5**	**6**	**1**	**2**	**3**	**4**	**5**	**6**
0	$T^{(1)}_0$	$T^{(2)}_0$	$T^{(3)}_0$	$T^{(4)}_0$	$T^{(5)}_0$	$T^{(6)}_0$	$\Sigma T^{(1)}_{0:0}$	$\Sigma T^{(2)}_{0:0}$	$\Sigma T^{(3)}_{0:0}$	$\Sigma T^{(4)}_{0:0}$	$\Sigma T^{(5)}_{0:0}$	$\Sigma T^{(6)}_{0:0}$
1	$T^{(1)}_1$	$T^{(2)}_1$	$T^{(3)}_1$	$T^{(4)}_1$	$T^{(5)}_1$	$T^{(6)}_1$	$\Sigma T^{(1)}_{0:1}$	$\Sigma T^{(2)}_{0:1}$	$\Sigma T^{(3)}_{0:1}$	$\Sigma T^{(4)}_{0:1}$	$\Sigma T^{(5)}_{0:1}$	$\Sigma T^{(6)}_{0:1}$
2	$T^{(1)}_2$	$T^{(2)}_2$	$T^{(3)}_2$	$T^{(4)}_2$	$T^{(5)}_2$	$T^{(6)}_2$	$\Sigma T^{(1)}_{0:2}$	$\Sigma T^{(2)}_{0:2}$	$\Sigma T^{(3)}_{0:2}$	$\Sigma T^{(4)}_{0:2}$	$\Sigma T^{(5)}_{0:2}$	$\Sigma T^{(6)}_{0:2}$
3	$T^{(1)}_3$	$T^{(2)}_3$	$T^{(3)}_3$	$T^{(4)}_3$	$T^{(5)}_3$	$T^{(6)}_3$	$\Sigma T^{(1)}_{0:3}$	$\Sigma T^{(2)}_{0:3}$	$\Sigma T^{(3)}_{0:3}$	$\Sigma T^{(4)}_{0:3}$	$\Sigma T^{(5)}_{0:3}$	$\Sigma T^{(6)}_{0:3}$
…	…	…	…	…	…	…	…	…	…	…	…	…
9	$T^{(1)}_9$	$T^{(2)}_9$	$T^{(3)}_9$	$T^{(4)}_9$	$T^{(5)}_9$	$T^{(6)}_9$	$\Sigma T^{(1)}_{0:9}$	$\Sigma T^{(2)}_{0:9}$	$\Sigma T^{(3)}_{0:9}$	$\Sigma T^{(4)}_{0:9}$	$\Sigma T^{(5)}_{0:9}$	$\Sigma T^{(6)}_{0:9}$

Example (for any permitted process count p): the cumulative MP time through G_2 is

$$\Sigma_{r=0}^{2} T^p(G_r) = T^p(G_0) + T^p(G_1) + T^p(G_2)$$

[D-n] Determine the speed-up

[D-n.1] For G_0, define the speed-up after completing k pairs with p processes as:

$$S_p(k) = T_k / T_k^p, \quad k = 1, \ldots, 8$$

For a given (p, k), obtain T_k from the measured cumulative sequential-time column of Table B-n.1, and $T_k^{(p)}$ from the measured cumulative multiprocessing-time column for process count p in Table C-n.1.

Example for p = 3 and k = 8:

$$S_3(8) = T_8 / T_8^3$$

Plot (i) speed-up versus process count using the complete G_0 workload (k = 8), and (ii) speed-up versus workload size k = 1, …, 8, with one curve for each permitted process count. Discuss the observed trends and bottlenecks in detail.

[D-n.2] For the all-group benchmark, determine speed-up from the cumulative group results in Tables B-n.2 and C-n.2 for G_0 through G_9.

[E] Compare the algorithms

Use the results obtained by all team members and present a detailed comparison of the various algorithms in the report.

[F-n] Intuitive Implementation Walkthrough

In addition to reporting experimental measurements, each team member must prepare a 1-page individual walkthrough titled "[F-n] Intuitive Implementation Walkthrough – <your name>" (appended to each member's individual section; not counted in the 12-page limit).

In your own words, using simple, clear, and layman-accessible language (as if explaining to a non-technical peer or junior student):

1. Core Algorithm Mechanism: Explain how your assigned algorithm actually computes matrix multiplication mechanically (use intuitive analogies or step-by-step visual metaphors rather than copying textbook mathematical formulas).

2. Workload Partitioning: Explain how your code divides the workload among Python worker processes (what each process computes and how intermediate results are combined).

3. Bottleneck Analysis: Explain in plain English why your algorithm does not achieve linear speed-up when adding more processes (where the overhead actually occurs, e.g. IPC data passing, process creation, CPU cache contention).

Note: Marks (5 pts) reward genuine conceptual mastery and clear, original articulation. Explain the concepts in your own words; do not rely on dense jargon, formula-heavy exposition, or generic/boilerplate definitions. If you have directly or partly adapted external baseline code, cite the source clearly here.

IV. Submission

What to submit?

Each group should upload **only one compressed file (.zip)** to *Canvas*, including three types of files: the report (.pdf), the source code file(s) (.py), and the readme text file (.txt).

Follow the rules below to name your files:

• For the compressed file:

CA2_(Group ID)_(Matriculation number of all team members separated by an underscore).zip

Example: CA2_12_A0213331Z_A0010101Y_A0340127X_A0711001X.zip

• For the report:

CA2_Report_(Group ID)_(Matriculation number of all team members separated by an underscore).pdf

Example: CA2_Report_12_A0213331Z_A0010101Y_A0340127X_A0711001X.pdf

Report format

Report Formatting Guidelines – Use single-column format. The main body font must be Times New Roman, 12 pt.

Page Limit – Maximum 12 pages (excluding the first page and the last page)

First Page of your report – Title + Full Names + Matriculation numbers (this page is not counted in the report page limit mentioned below).

Task [F-n] Pages – Each member must include their 1-page Intuitive Implementation Walkthrough as specified in Section III [F-n] (not counted in the 12-page main report limit).

Code file(s) – Include useful comments throughout the code so readers can understand the computation.

Readme file

This file must contain clear instructions to facilitate running your program and use of data and parameters. If you are using any specific packages, list them and also indicate the URLs from where the readers can download. If you are using any specific DATA from any URL, indicate the URL. If the readers need to set any input parameters (hard coded way) explicitly state that requirement. Do NOT ASSUME anything and omit details. Step-by-step instructions will be very useful.

V. Assessment

Grading

Your assignment will be graded out of 60 marks, and the final weight of this assignment is **35%.** For the joint effort, all members will be awarded identical marks. The mark breakdown is shown in Table III.

Table III. Grade breakdown

Joint effort (50%)	Effort in (A) and (E). Title, abstract, introduction, joint

	discussions, and conclusion.
Individual effort (50%)	Effort in (B), (C), (D), and (F). Sections on interpreting individual results and intuitive walkthrough.

Plagiarism Penalty

You are allowed to use only 15% of any written material collectively from all other sources. Anything more than this 15% will be penalized. References cited may show up in your plagiarism checker and you can ignore this percentage. **Copying of coding and simulation results directly from others will be awarded 0 marks for both Source and Copier(s)**.

Rubrics

Criteria	Remarks	Points
[Joint] Abstract	Capture the algorithms and implementation approaches used and highlight of any significant results in not more than 12 lines.	2
[Joint] Introduction	Brief introduction of the algorithms and implementation approaches. Clear problem statement in plain English and also using technical description.	5
[Joint] Description of input data [A]	Describe the input matrix set and how it is generated. Identify any special packages clearly.	6
[Indiv] Description of algorithm and implementation approach [B, C]	Each one of you can open a separate sub-section in this part. Describe the algorithms in a concise fashion. Highlight any non-trivial decisions/steps that algorithm takes and use proper citations. You may use a simple numerical example to explain, if you wish. Describe clearly how you implement the multi-processing method. Elaborate how you identify the bottleneck and maximize the parallelism and acceleration enabled by the multi-processing approach.	10
[Indiv] Individual results and discussions [B, C]	Each one of you can open a separate sub-section in this part. Put all your simulation results in a systematic fashion under each subsection, point to the results and argue on the trends, behaviour, results, etc, clearly. Clearly interpret and discuss your table / graphs and try to relate to algorithms described in the earlier section.	15
[Joint] Joint discussions [E]	This is the joint discussion under Task E. You may wish to open a common section and put your arguments/discussions/comments as a comparison of the algorithms, if it is valid. This demonstrates your group effort.	12

	Describe strengths and pitfalls of the algorithms, if any, clearly. Argue why algo(s) fails in certain conditions and what it does not consider, etc. You may describe here what you feel is right and wrong that is within the scope of formulations used in the paper.	
[Joint] Conclusions & reference(s) used [E]	Conclude how the algorithms benefits/suffers from the multi-processing, and the potentials/limitations to the number of processes/workloads in not more than 12 lines. Formally list the paper(s) & sites you have directly used.	5
[Indiv] Intuitive implementation walkthrough [F-n]	Explain the core algorithm mechanics, multiprocessing workload division, and bottleneck analysis in clear, layman-accessible language. Demonstrate genuine conceptual mastery through clear, original articulation rather than dense jargon, formula-heavy exposition, or generic/boilerplate definitions. (1 page per member.)	5
TOTAL MARKS		**60**

Appendix A

The following gives the general ideas of four common algorithms used to perform matrix multiplication. Here we consider $Z = Y \cdot X$.

1. Strassen's algorithm

Divide matrices X, Y, and Z into four parts,

$$\begin{bmatrix} Z_{11} & Z_{21} \\ Z_{21} & Z_{22} \end{bmatrix} = \begin{bmatrix} X_{11} & X_{12} \\ X_{21} & X_{22} \end{bmatrix} \cdot \begin{bmatrix} Y_{11} & Y_{12} \\ Y_{21} & Y_{22} \end{bmatrix}$$

Compute the following,

$$M_1 = (X_{11} + X_{21}) \cdot (Y_{11} + Y_{12})$$
$$M_2 = (X_{12} + X_{22}) \cdot (Y_{21} + Y_{22})$$
$$M_3 = (X_{11} - X_{22}) \cdot (Y_{11} + Y_{22})$$
$$M_4 = X_{11} \cdot (Y_{12} - Y_{22})$$
$$M_5 = (X_{21} + X_{22}) \cdot Y_{11}$$
$$M_6 = (X_{11} + X_{12}) \cdot (Y_{22})$$
$$M_7 = X_{22} \cdot (Y_{21} - Y_{11})$$

Finally,

$$Z_{11} = M_2 + M_3 - M_6 - M_7$$
$$Z_{12} = M_4 + M_6$$
$$Z_{21} = M_5 + M_7$$
$$Z_{22} = M_1 - M_3 - M_4 - M_5$$

2. Coppersmith–Winograd algorithm

Divide matrices X, Y, and Z into four parts,

$$\begin{bmatrix} Z_{11} & Z_{12} \\ Z_{21} & Z_{22} \end{bmatrix} = \begin{bmatrix} X_{11} & X_{12} \\ X_{21} & X_{22} \end{bmatrix} \cdot \begin{bmatrix} Y_{11} & Y_{12} \\ Y_{21} & Y_{22} \end{bmatrix}$$

Compute the following,

$$S_1 = X_{21} + X_{22}$$
$$S_2 = S_1 - X_{11}$$
$$S_3 = X_{11} - X_{21}$$
$$S_4 = X_{12} + S_2$$

$$T_1 = Y_{12} - Y_{11}$$
$$T_2 = Y_{22} - T_1$$
$$T_3 = Y_{22} - Y_{12}$$
$$T_4 = T_2 - Y_{21}$$

$$M_1 = X_{11} \cdot Y_{11}$$
$$M_2 = X_{12} \cdot Y_{21}$$
$$M_3 = S_4 \cdot Y_{22}$$
$$M_4 = X_{22} \cdot T_4$$
$$M_5 = S_1 \cdot T_1$$
$$M_6 = S_2 \cdot T_2$$
$$M_7 = S_3 \cdot T_3$$

Finally,

$$Z_{11} = M_1 + M_2$$
$$Z_{12} = M_1 + M_3 + M_5 + M_6$$
$$Z_{21} = M_7 - M_5$$
$$Z_{12} = M_1 + M_5 + M_6 + M_7$$

3. Cannon's algorithm

Here we divide matrices X, Y, and Z into nine parts for example, it will take three stages to perform the multiplication. Note that $\circ$ stand for element-wise product.

$$\begin{bmatrix} Z_{11} & Z_{12} & Z_{13} \\ Z_{21} & Z_{22} & Z_{23} \\ Z_{31} & Z_{32} & Z_{33} \end{bmatrix} = \begin{bmatrix} X_{11} & X_{12} & X_{13} \\ X_{21} & X_{22} & X_{23} \\ X_{31} & X_{32} & X_{33} \end{bmatrix} \cdot \begin{bmatrix} Y_{11} & Y_{12} & Y_{13} \\ Y_{21} & Y_{22} & Y_{23} \\ Y_{31} & Y_{32} & Y_{33} \end{bmatrix}$$

In stage 1,

$$M_1 = \begin{bmatrix} X_{11} & X_{12} & X_{13} \\ X_{22} & X_{23} & X_{21} \\ X_{33} & X_{31} & X_{32} \end{bmatrix} \circ \begin{bmatrix} Y_{11} & Y_{22} & Y_{33} \\ Y_{21} & Y_{32} & Y_{13} \\ Y_{31} & Y_{12} & Y_{23} \end{bmatrix} = \begin{bmatrix} X_{11}Y_{11} & X_{12}Y_{22} & X_{13}Y_{33} \\ X_{22}Y_{21} & X_{23}Y_{32} & X_{21}Y_{13} \\ X_{33}Y_{31} & X_{31}Y_{12} & X_{32}Y_{23} \end{bmatrix}$$

In stage 2,

$$M_2 = \begin{bmatrix} X_{12} & X_{13} & X_{11} \\ X_{23} & X_{21} & X_{22} \\ X_{31} & X_{32} & X_{33} \end{bmatrix} \circ \begin{bmatrix} Y_{21} & Y_{32} & Y_{13} \\ Y_{31} & Y_{12} & Y_{23} \\ Y_{11} & Y_{22} & Y_{33} \end{bmatrix}$$

Stage 3,

$$M_3 = \begin{bmatrix} X_{13} & X_{11} & X_{12} \\ X_{21} & X_{22} & X_{23} \\ X_{32} & X_{33} & X_{31} \end{bmatrix} \circ \begin{bmatrix} Y_{31} & Y_{12} & Y_{23} \\ Y_{11} & Y_{22} & Y_{33} \\ Y_{21} & Y_{32} & Y_{13} \end{bmatrix}$$

Finally,

$$Z = M_1 + M_2 + M_3$$

4. Fox's algorithm

Here we divide matrices X, Y, and Z into nine parts for example, it will take three stages to perform the multiplication.

$$\begin{bmatrix} Z_{11} & Z_{12} & Z_{13} \\ Z_{21} & Z_{22} & Z_{23} \\ Z_{31} & Z_{32} & Z_{33} \end{bmatrix} = \begin{bmatrix} X_{11} & X_{12} & X_{13} \\ X_{21} & X_{22} & X_{23} \\ X_{31} & X_{32} & X_{33} \end{bmatrix} \cdot \begin{bmatrix} Y_{11} & Y_{12} & Y_{13} \\ Y_{21} & Y_{22} & Y_{23} \\ Y_{31} & Y_{32} & Y_{33} \end{bmatrix}$$

In stage 1,

$$M_1 = \begin{bmatrix} X_{11} & X_{12} & X_{13} \\ X_{21} & X_{22} & X_{23} \\ X_{31} & X_{32} & X_{33} \end{bmatrix} \circ \begin{bmatrix} Y_{11} & Y_{22} & Y_{33} \\ Y_{11} & Y_{22} & Y_{33} \\ Y_{11} & Y_{22} & Y_{33} \end{bmatrix} = \begin{bmatrix} X_{11}Y_{11} & X_{12}Y_{22} & X_{13}Y_{33} \\ X_{21}Y_{11} & X_{22}Y_{22} & X_{23}Y_{33} \\ X_{31}Y_{11} & X_{32}Y_{22} & X_{33}Y_{33} \end{bmatrix}$$

In stage 2,

$$M_2 = \begin{bmatrix} X_{12} & X_{13} & X_{11} \\ X_{22} & X_{23} & X_{21} \\ X_{32} & X_{33} & X_{31} \end{bmatrix} \circ \begin{bmatrix} Y_{21} & Y_{32} & Y_{13} \\ Y_{21} & Y_{32} & Y_{13} \\ Y_{21} & Y_{32} & Y_{13} \end{bmatrix}$$

In stage 3,

$$M_3 = \begin{bmatrix} X_{13} & X_{11} & X_{12} \\ X_{23} & X_{21} & X_{22} \\ X_{33} & X_{31} & X_{32} \end{bmatrix} \circ \begin{bmatrix} Y_{31} & Y_{12} & Y_{23} \\ Y_{31} & Y_{12} & Y_{23} \\ Y_{31} & Y_{12} & Y_{23} \end{bmatrix}$$

Finally,

$$Z = M_1 + M_2 + M_3$$

References [*If you wish to do any additional reading you may use these references*]

[1] V. Strassen, "Gaussian elimination is not optimal," Numer Math (Heidelb), vol. 13, no. 4, pp. 354–356, 1969, doi: 10.1007/BF02165411.

[2] D. Coppersmith and S. Winograd, "Matrix multiplication via arithmetic progressions," J Symb Comput, vol. 9, no. 3, pp. 251–280, 1990, doi: https://doi.org/10.1016/S0747-7171(08)80013-2.

[3] L. E. Cannon, "A Cellular Computer to Implement the Kalman Filter Algorithm," PhD Thesis, Montana State University, Bozeman, Montana, USA, 1969, [Online]. Available: https://scholarworks.montana.edu/xmlui/bitstream/handle/1/4168/31762100054244.pdf?sequence=1.

[4] G. C. Fox, S. W. Otto, and A. J. G. Hey, "Matrix algorithms on a hypercube I: Matrix multiplication," Parallel Comput, vol. 4, no. 1, pp. 17–31, 1987, doi: https://doi.org/10.1016/0167-8191(87)90060-3.

----- END -----